

DOCUMENTAÇÃO DO SISTEMA

Assistência Técnica

Projeto .NET MAUI com MVC, Services, DAO, MySQL, MongoDB, SQLite, exportação e importação JSON e uso de gráficos.

São José dos Campos – SP
2026

Emily Colen Honorio – 02320114 – 7UNA

Link GitHub: https://github.com/Emily-Colen/AssistenciaTecnica_APPGestao

Link Demonstração YouTube: <https://youtu.be/TWEVRRPNRZg>

Sumário

1. Tema escolhido
 2. Objetivo
 3. Regras de negócio
 4. Estrutura MVC
 5. Explicação dos Services
 6. Estrutura do banco de dados
 7. Explicação do MongoDB
 8. Explicação do SQLite
 9. Exportação e importação JSON
 10. Relatórios e gráficos
 11. Tecnologias e pacotes utilizados
 12. Conclusão
- Apêndice A — Relação resumida de arquivos

1. Tema escolhido

O tema escolhido para o projeto é um sistema de gestão para assistência técnica. A aplicação, chamada Assistência Técnica, tem como finalidade controlar clientes, equipamentos, técnicos, ordens de serviço, peças, estoque, anexos e backups de dados em um único sistema.

O projeto foi desenvolvido em .NET MAUI, com organização em camadas e integração com diferentes tipos de armazenamento: MySQL para os dados principais, MongoDB para documentos/anexos e SQLite para cache local do aplicativo.

2. Objetivo

O objetivo do sistema é oferecer uma aplicação desktop/mobile capaz de organizar o fluxo de atendimento de uma assistência técnica, desde o cadastro do cliente até o acompanhamento de ordens de serviço e controle de peças utilizadas.

A proposta também atende aos requisitos acadêmicos de separação de responsabilidades, utilização de banco relacional com relacionamentos, uso de banco não relacional, cache local, exportação de dados e apresentação de indicadores gráficos.

Objetivo específico	Descrição
Controle de clientes	Cadastrar, listar, atualizar e excluir clientes com dados básicos e endereço.
Controle de peças	Gerenciar peças, valores, estoque atual e estoque mínimo.
Controle de ordens de serviço	Abrir ordens, alterar status, vincular equipamento, técnico e peças utilizadas.
Backup de dados	Exportar informações em arquivo JSON e validar arquivos JSON importados.
Indicadores	Exibir na tela principal um gráfico simples de ordens agrupadas por status.

3. Regras de negócio

As regras de negócio estão concentradas principalmente nas classes de Service e nos Helpers de validação. Essa organização evita que as telas acessem diretamente o banco de dados e garante que os dados sejam conferidos antes das operações de inserção, alteração ou exclusão.

Área	Regra aplicada
Login	O técnico deve informar e-mail e senha. O sistema verifica existência do usuário, status ativo e senha por hash SHA-256.
Cliente	Nome, CPF/CNPJ, e-mail, telefone e rua são obrigatórios. O cliente precisa estar associado a uma rua válida.
Peça	Nome obrigatório, valores de custo e venda não negativos, estoque atual e estoque mínimo não negativos.
Estoque	Entradas e saídas exigem peça válida, quantidade positiva e valor unitário não negativo. Saídas podem ser vinculadas a uma ordem de serviço.
Ordem de serviço	A ordem precisa ter defeito relatado, equipamento válido, status válido e preço não negativo.
Peças na OS	Ao adicionar peça em uma ordem, o sistema valida peça, quantidade, valor unitário e calcula o subtotal.
Backup JSON	O arquivo importado precisa existir, possuir extensão .json e conter dados válidos para clientes, peças e ordens.

4. Estrutura MVC

O projeto utiliza uma organização próxima ao padrão MVC, acrescentando Services, DAOs e Interfaces para separar responsabilidades. Essa divisão facilita manutenção, testes e evolução do sistema.

Camada	Pasta/arquivos	Responsabilidade
Model	Models	Representa as entidades do domínio, como Cliente, Técnico, Peça, Equipamento e OrdemServico.
View	Views	Contém as telas XAML e seus arquivos code-behind, como LoginPage, MainMenuPage, ClientePage, PecaPage e OrdemServicoPage.
Controller	Controllers	Recebe chamadas da View e encaminha para os Services, mantendo a tela mais limpa.
Service	Services	Concentra regras de negócio, validações e chamadas para a camada DAO.
DAO	DAO	Executa consultas e comandos no banco de dados.
Interfaces	Interfaces	Define contratos simples para DAOs, Services e Controllers.
Database	Database	Centraliza configurações e factories de conexão com MySQL, MongoDB e SQLite.

Fluxo geral: o usuário interage com a View; a View chama o Controller ou Service; o Service valida os dados; o DAO executa a operação no banco; o resultado retorna para a tela.

5. Explicação dos Services

Os Services são a camada mais importante para as regras de negócio. Eles fazem a ponte entre as telas/controladores e os DAOs, centralizando validações e evitando duplicação de lógica.

Service	Função no sistema
LoginService	Realiza autenticação do técnico, valida campos obrigatórios, verifica usuário ativo e compara senha com hash.
TecnicoService	Permite criar e listar técnicos, gerando hash da senha antes da gravação.
ClienteService	Lista, insere, atualiza e exclui clientes, validando dados obrigatórios.
EquipamentoService	Lista equipamentos por cliente e insere novos equipamentos vinculados a clientes.
StatusOrdemServicoService	Lista os status disponíveis para ordens de serviço.
OrdemServicoService	Controla abertura de OS, atualização de status e inclusão de peças na ordem.
PecaService	Gerencia peças e valida valores e estoque.
EstoqueService	Registra entrada e saída de peças, inclusive saída vinculada a ordem de serviço.
JsonBackupService	Gera arquivo JSON de backup, lê arquivo JSON selecionado e valida seu conteúdo.
MongoAnexoService	Salva e lista anexos/documentos vinculados a ordens de serviço no MongoDB.
LocalCacheService	Inicializa o SQLite local e salva cache simplificado de peças.

6. Estrutura do banco de dados

O banco principal configurado no projeto é o MySQL, com nome assistencia_tecnica. O acesso é feito com MySqlConnection e Dapper por meio da classe MySqlConnectionFactory. Os Models indicam as entidades principais e os DAOs executam as consultas SQL.

Tabela/Entidade	Finalidade
estado, cidade, bairro, rua	Estrutura de endereço usada no cadastro de clientes.
cliente	Armazena dados do cliente, como nome, CPF/CNPJ, e-mail e telefone.
tecnico	Armazena usuários técnicos que podem acessar o sistema e atender ordens.
especialidade	Representa áreas de atuação técnica.
tecnico_has_especialidade	Tabela intermediária para relacionamento N:N entre técnico e especialidade.
equipamento	Armazena equipamentos pertencentes aos clientes.
status_ordem_servico	Define os status possíveis para uma ordem de serviço.
ordem_servico	Registra as ordens abertas para equipamentos, com defeito, preço, técnico e status.
peca	Controla peças, valores e estoque.
peca_has_ordem_servico	Tabela intermediária N:N entre peças e ordens de serviço.
movimentacao_estoque	Registra entradas e saídas de peças.
garantia	Armazena informações de garantia relacionadas a ordens de serviço.

6.1 Relacionamentos principais

- Estado → Cidade → Bairro → Rua → Cliente: estrutura hierárquica de endereço.
- Cliente → Equipamento: um cliente pode possuir vários equipamentos.
- Equipamento → Ordem de Serviço: uma OS é aberta para um equipamento específico.
- Técnico → Ordem de Serviço: uma OS pode possuir técnico responsável.
- Status → Ordem de Serviço: cada OS possui um status, como aberta, em andamento ou finalizada.
- Peça ↔ Ordem de Serviço: relacionamento N:N controlado por peca_has_ordem_servico.
- Técnico ↔ Especialidade: relacionamento N:N controlado por tecnico_has_especialidade.
- Peça → Movimentação de Estoque: cada entrada ou saída gera registro histórico.

7. Explicação do MongoDB

O MongoDB é utilizado como banco documental para anexos e documentos vinculados às ordens de serviço. Essa escolha é adequada porque anexos e metadados podem variar em formato, tamanho e quantidade, o que combina melhor com uma base documental.

A conexão é configurada em DatabaseConfig, com MongoClient e MongoDatabaseName. A classe MongoClientFactory cria o acesso ao banco, e o MongoAnexoDao utiliza a coleção anexos_ordem_servico para salvar e consultar documentos.

Elemento	Descrição
AnexoMongo	DTO que representa o anexo/documento associado à ordem de serviço.
MongoAnexoDao	Classe responsável por inserir anexos e listar anexos por ordem de serviço.
MongoAnexoService	Camada de regra de negócio que valida a ordem antes de salvar ou listar anexos.
Coleção	anexos_ordem_servico.

Os campos usados para o anexo incluem Id, OrdemServicoid, Tecnicoid, NomeArquivo, TipoArquivo, CaminhoArquivo, Descricao e DataUpload. Dessa forma, o sistema pode associar documentos a uma OS sem sobrecarregar o banco relacional.

8. Explicação do SQLite

O SQLite é utilizado como banco local do aplicativo. O caminho é definido em DatabaseConfig por meio de FileSystem.AppDataDirectory, gerando o arquivo assistencia_local.db3 dentro da área de dados do aplicativo.

A classe SQLiteConnectionFactory cria a conexão local, enquanto o LocalCacheDao cria a tabela PecaCache e salva uma cópia simplificada das peças. O LocalCacheService expõe as operações de inicialização e gravação do cache.

Componente	Função
assistencia_local.db3	Arquivo local SQLite criado na área de dados do aplicativo.
PecaCache	Tabela local com IdPeca, Nome e EstoqueAtual.
LocalCacheDao	Cria a tabela e atualiza o cache local de peças.
LocalCacheService	Organiza a chamada da camada DAO para o cache local.

9. Exportação e importação JSON

A exportação de dados do projeto é realizada em formato JSON. Ao clicar em Exportar JSON na tela principal, o sistema gera um arquivo de backup no padrão backup_assistencia_yyyyMMdd_HHmms.json dentro do diretório FileSystem.AppDataDirectory.

O JsonBackupService chama o BackupDao, que consulta as tabelas principais do MySQL e monta um objeto BackupJson. Em seguida, esse objeto é serializado com System.Text.Json utilizando indentação para facilitar leitura e conferência.

Etapa	Descrição
1. Clique em Exportar JSON	Evento OnExportarJsonClicked é executado na MainMenuPage.
2. Geração do caminho	O nome do arquivo é criado com data e hora para evitar sobrescrita.
3. Consulta dos dados	BackupDao busca estados, cidades, clientes, técnicos, ordens, peças, estoque e garantias.
4. Serialização	JsonBackupService converte o objeto BackupJson para texto JSON formatado.
5. Gravação	O arquivo é salvo no diretório local do aplicativo.

Na importação, o usuário seleciona um arquivo pelo FilePicker. O sistema verifica se o arquivo existe, se possui extensão .json e se o conteúdo é válido. A validação impede, por exemplo, cliente sem nome, peça com estoque negativo ou ordem de serviço sem defeito relatado.

10. Relatórios e gráficos

O projeto possui dados suficientes para gerar relatórios administrativos e indicadores. Como melhoria implementada na tela principal, foi adicionado um gráfico simples de barras para exibir a quantidade de ordens de serviço agrupadas por status.

O gráfico é construído diretamente com componentes nativos do .NET MAUI, como Border, Grid, Label, Button e BoxView. Dessa forma, não é necessário instalar biblioteca externa de gráficos. A tela principal chama OrdemServicoService para listar as ordens e StatusOrdemServicoService para identificar o nome de cada status.

Indicador/relatório	Origem dos dados	Finalidade
Ordens por status	ordem_servico + status_ordem_servico	Mostrar rapidamente quantas ordens estão abertas, em andamento ou finalizadas.
Estoque por peça	peca	Acompanhar estoque atual e estoque mínimo.
Movimentações de estoque	movimentacao_estoque	Analisar entradas e saídas de peças.
Histórico de atendimento	ordem_servico + cliente + equipamento	Acompanhar serviços prestados por cliente/equipamento.

Na documentação do código, a funcionalidade do gráfico pode ser descrita como um relatório visual simplificado no dashboard, permitindo ao usuário identificar a situação geral das ordens sem precisar abrir a tela de ordens de serviço.

11. Tecnologias e pacotes utilizados

Tecnologia/pacote	Uso no projeto
.NET MAUI	Desenvolvimento da interface multiplataforma, com foco em execução no Windows.
C#	Linguagem principal da aplicação.
XAML	Construção das telas do aplicativo.
MySQL	Banco relacional principal.
MySqlConnection	Conexão com o banco MySQL.
Dapper	Mapeamento simples entre SQL e objetos C#.
MongoDB.Driver	Acesso ao MongoDB para anexos/documentos.
sqlite-net-pcl	Criação e acesso ao banco SQLite local.
System.Text.Json	Serialização e leitura de arquivos JSON.
Microsoft.Extensions.DependencyInjection	Injeção de dependência no MauiProgram.cs.

12. Conclusão

O projeto para Assistência Técnica apresenta uma estrutura organizada e coerente para aplicação de gestão e auxiliou fortemente o entendimento acadêmico. A separação entre Models, Views, Controllers, Services, DAOs, Interfaces e classes de conexão facilitam manutenção e demonstra entendimento da arquitetura em camadas.

O MySQL é usado como banco principal relacional, o MongoDB é reservado para anexos e documentos de ordens de serviço, e o SQLite é utilizado como cache local. A exportação e validação de backup em JSON reforçam a portabilidade dos dados, enquanto o gráfico simples na tela principal melhora a visualização dos indicadores do sistema.

Com essas características, o sistema atende aos principais requisitos propostos: uso de múltiplos bancos, organização MVC, camada de Services, validações, controle de ordens de serviço, estoque, backup e apresentação de gráficos simples.

Apêndice A — Relação resumida de arquivos

Pasta/arquivo	Descrição
Models	Classes de domínio do sistema.
Views	Telas XAML e code-behind do aplicativo.
Controllers	Controladores de autenticação, clientes, ordens e peças.
Services	Camada de regra de negócio e validação.
DAO	Acesso ao banco de dados e consultas SQL/Mongo/SQLite.
Database	Configurações e factories de conexão.

DTOs	Objetos de transferência, como BackupJson, LoginResultado e AnexoMongo.
Helpers	Funções auxiliares de senha e validação.
MauiProgram.cs	Registro de dependências e configuração inicial do app.
MainMenuPage	Tela principal com navegação, backup JSON e gráfico simples de indicadores.